ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ
«КАЗАНСКИЙ (ПРИВОЛЖСКИЙ) ФЕДЕРАЛЬНЫЙ
УНИВЕРСИТЕТ»

ИНСТИТУТ ВЫЧИСЛИТЕЛЬНОЙ МАТЕМАТИКИ
И ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ

ОТЧЕТ

по проекту

на тему «Многослойный персептрон»

Работу выполнила студентка 4 курса 09-321 группы
Кириллова Дарья Евгеньевна

Преподаватель: Гиниятова Динара Халиловна

г. Казань 2026 г.

ЦЕЛЬ ПРОЕКТА

Цель проекта— сделать первый шаг в изучении искусственных нейронных сетей и реализовать «с нуля» алгоритмы, лежащие в основе их обучения: прямое распространение, обратное распространение ошибки и градиентный спуск. Для этого требуется построить многослойный персептрон с не менее чем двумя скрытыми слоями и обучить его решать задачу бинарной классификации: по 30 числовым характеристикам ядер клеток молочной железы, полученным при тонкоигольной биопсии, определить, является опухоль злокачественной (M) или доброкачественной (B).

Использование библиотек, реализующих нейронные сети (Keras, PyTorch и т. п.), запрещено, поэтому вся математика сети написана на `numpy`. Проект закрепляет вычисление производной сложной функции и основы линейной алгебры, а также навыки подготовки данных: анализ, стандартизация, разделение на обучающую и проверочную выборки.

ХОД РАБОТЫ

Задание состоит из трёх независимых программ: разделение данных (`split_data.py`), обучение (`train.py`) и прогнозирование (`predict.py`). Класс сети вынесен в отдельный модуль `neural_network.py`, который импортируется двумя последними программами.

Этап 1. Подготовка данных (`split_data.py`)

Программа выполняет следующие шаги:

1. читает `data.csv` без заголовка и присваивает имена столбцам, отбрасывает идентификатор (он не несёт информации о диагнозе);

2. кодирует метку в виде one-hot вектора: $M \mapsto (1, 0)$, $B \mapsto (0, 1)$— именно такой формат нужен выходному слою softmax из двух нейронов;

3. делит выборку на обучающую (80 %, 455 объектов, из них 169 класса M) и проверочную (20 %, 114 объектов, из них 43 класса M) функцией `train_test_split` с фиксированным `random_state=42`, чтобы разбиение воспроизводилось;

4. стандартизует признаки: $x_{norm} = \dfrac{x - \mu}{\sigma}$, где среднее μ и стандартное отклонение σ считаются *только по обучающей выборке*, а затем применяются к проверочной. Так в модель не попадает информация о тестовых данных (утечка данных);

5. сохраняет результат в `train.csv` и `test.csv`.

После преобразования среднее каждого признака в обучающей выборке равно нулю (порядка 10^{-16}), а стандартное отклонение равно единице.

Этап 2. Архитектура сети и прямое распространение

Сеть описывается списком `layer_sizes`, поэтому число слоёв и нейронов настраивается произвольно. Архитектура по умолчанию— `[30, 64, 32, 2]`: входной слой из 30 признаков, два скрытых слоя (64 и 32 нейрона) и выходной слой из двух нейронов. Всего обучаемых параметров $64 \cdot 30 + 64 + 32 \cdot 64 + 32 + 2 \cdot 32 + 2 = 4130$.

Объекты хранятся по столбцам: матрица X имеет размер $30 \times m$, где m— число объектов. Для слоя l вычисляются

$$Z^{[l]} = W^{[l]} A^{[l-1]} + b^{[l]}, \qquad A^{[l]} = \mathrm{ReLU}(Z^{[l]}) = \max(0, Z^{[l]}), \quad A^{[0]} = X, \tag{1}$$

а на выходном слое применяется softmax, превращающий два числа в распределение вероятностей:

$$\hat{y}_k = \frac{e^{z_k}}{\sum_j e^{z_j}}. \tag{2}$$

В коде из каждого z предварительно вычитается максимум по столбцу: результат не меняется, но исключается переполнение `exp`. Веса инициализируются по Хе: $W^{[l]} \sim \mathcal{N}\big(0,\, 2/n_{l-1}\big)$, смещения— нулями. Такая инициализация сохраняет дисперсию сигнала при ReLU и ускоряет начало обучения.

```
def forward(self, x):
    cache = {}; A = x
    for l in range(1, self.num_layers):          # hidden layers
        cache[f'A{l-1}'] = A
        z = self.params[f'W{l}'] @ A + self.params[f'b{l}']
        cache[f'z{l}'] = z; A = self.relu(z); cache[f'A{l}'] = A
    l = self.num_layers                          # output layer
    cache[f'A{l-1}'] = A
    z = self.params[f'W{l}'] @ A + self.params[f'b{l}']
    cache[f'z{l}'] = z
    A_last = self.softmax(z); cache[f'A{l}'] = A_last
    return A_last, cache
```

Этап 3. Функция потерь и обратное распространение

Качество предсказания измеряется кросс-энтропией, которая для двух классов совпадает с бинарной кросс-энтропией:

$$J = -\frac{1}{m} \sum_{i=1}^{m} \sum_{k=1}^{2} y_k^{(i)} \ln \hat{y}_k^{(i)}. \tag{3}$$

Перед логарифмом предсказания обрезаются до отрезка $[10^{-15},\, 1 - 10^{-15}]$, чтобы избежать $\ln 0$.

Градиенты находятся по правилу дифференцирования сложной

функции, начиная с выхода. Для связки softmax и кросс-энтропии производная по входу выходного слоя упрощается до разности предсказания и истины:

$$dZ^{[L]} = A^{[L]} - Y. \tag{4}$$

Далее для каждого слоя $l = L, \ldots, 1$:

$$dW^{[l]} = \frac{1}{m}\, dZ^{[l]} \big(A^{[l-1]}\big)^T, \qquad db^{[l]} = \frac{1}{m} \sum_i dZ^{[l]}_{\cdot i}, \tag{5}$$

$$dZ^{[l-1]} = \big(W^{[l]}\big)^T dZ^{[l]} \odot \mathbb{1}\Big[Z^{[l-1]} > 0\Big], \tag{6}$$

где $\odot$ — поэлементное произведение, а индикатор $\mathbb{1}[Z > 0]$ — производная ReLU. Значения $Z^{[l]}$ и $A^{[l]}$ берутся из кэша прямого прохода, поэтому повторных вычислений нет.

```
dz = cache[f'A{L}'] - y_true                          # softmax + cross-entropy
grads[f'dW{L}'] = (1/m) * dz @ cache[f'A{L-1}'].T
grads[f'db{L}'] = (1/m) * np.sum(dz, axis=1, keepdims=True)
for l in range(L-1, 0, -1):
    dz = self.params[f'W{l+1}'].T @ dz * self.relu_derivative(cache[f'z{l}'])
    grads[f'dW{l}'] = (1/m) * dz @ cache[f'A{l-1}'].T
    grads[f'db{l}'] = (1/m) * np.sum(dz, axis=1, keepdims=True)
```

Этап 4. Градиентный спуск и обучение (`train.py`)

Градиентный спуск движется в сторону, противоположную градиенту функции потерь, одновременно для всех параметров:

$$W^{[l]} \leftarrow W^{[l]} - \alpha\, dW^{[l]}, \qquad b^{[l]} \leftarrow b^{[l]} - \alpha\, db^{[l]}. \tag{7}$$

Здесь α — скорость обучения. Обучение полнопакетное (batch): на каждой эпохе выполняются прямой проход по всей обучающей выборке, подсчёт потерь, обратный проход и одно обновление весов. Использованы $\alpha = 0{,}005$ и 500 эпох. На каждой эпохе записываются потери и точность на обучающей выборке, а на проверочной — на первой и каждой десятой эпохе (при этом веса по проверочным данным не обновляются). В консоль выводятся строки вида `Эпоха 500/500 | Train Loss: 0.112370 | Train Acc: 96.3% | Test Loss: 0.084543 | Test Acc: 98.2%`. В конце обучения веса и архитектура сохраняются в файл `model_weights.npz`, а кривые обучения — в `training_curves.png`.

Потери монотонно убывают с 0,72 до 0,112 на обучающей и с 0,61 до 0,085 на проверочной выборке (рис. 1); признаков переобучения (рост проверочных потерь при падении обучающих) нет. Проверочная кривая лежит ниже обучающей, что нормально для небольшой выборки из 114

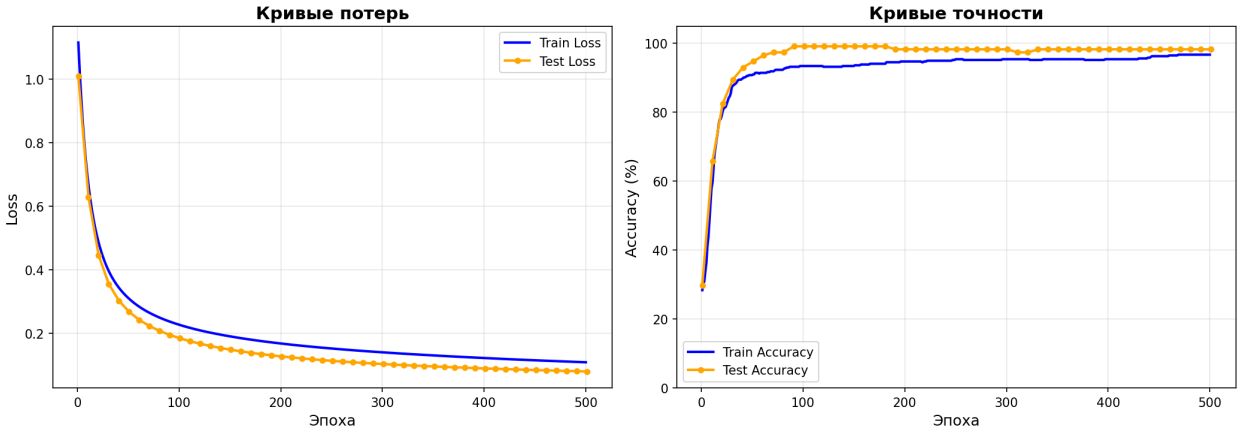


Рис. 1. Кривые потерь и точности на обучающей и проверочной выборках

объектов: проверочная выборка оказалась «легче», а одна ошибка меняет точность на 0,9 п. п. К 500-й эпохе потери ещё снижаются, то есть обучение не дошло до насыщения, и более высокая скорость обучения или большее число эпох улучшили бы результат на обучающих данных.

Этап 5. Прогнозирование и оценка (`predict.py`)

Программа загружает архитектуру и веса из `model_weights.npz`, прогоняет проверочную выборку через сеть, берёт класс с наибольшей вероятностью (`argmax`) и считает бинарную кросс-энтропию и точность. Результаты запуска:

Показатель	Обучающая выборка	Проверочная выборка
Кросс-энтропия	0,1124	0,0845
Точность	96,26 %	98,25 % (112 из 114)
Полнота по классу M	—	97,67 % (42 из 43)
Точность (precision) по классу M	—	97,67 % (42 из 43)

Таблица 1. Итоговые метрики модели `[30, 64, 32, 2]`

Сеть допустила две ошибки: один злокачественный случай классифицирован как доброкачественный (ложноотрицательный, $P(M) = 0{,}20$) и один доброкачественный— как злокачественный (ложноположительный, $P(M) = 0{,}66$). Для медицинской задачи опаснее ложноотрицательная ошибка, поэтому на практике порог принятия решения для класса M можно было бы снизить, жертвуя точностью ради полноты. Обе ошибочные вероятности далеки от уверенных значений 0 и 1, то есть сеть «понимает», что случай пограничный.

Этап 7. Влияние гиперпараметров

Начальные веса задаются случайно, поэтому результат одного запуска зависит от инициализации. Для честного сравнения каждая конфигурация обучалась 500 эпох на 10 разных начальных значениях генератора;

в таблице указаны среднее и стандартное отклонение.

Архитектура	Параметров	Test loss	Test acc., %	Train acc., %
`[30, 2]` (без скрытых)	62	$0{,}114 \pm 0{,}010$	$97{,}5 \pm 0{,}9$	$95{,}9 \pm 0{,}8$
`[30, 16, 2]`	530	$0{,}146 \pm 0{,}023$	$96{,}0 \pm 1{,}1$	$95{,}1 \pm 1{,}2$
`[30, 32, 16, 2]`	1554	$0{,}129 \pm 0{,}022$	$95{,}8 \pm 1{,}2$	$96{,}1 \pm 0{,}9$
`[30, 64, 32, 2]`	4130	$0{,}105 \pm 0{,}010$	$96{,}8 \pm 0{,}9$	$96{,}7 \pm 0{,}6$
`[30, 128, 64, 32, 2]`	14370	$0{,}087 \pm 0{,}011$	$96{,}8 \pm 0{,}7$	$97{,}6 \pm 0{,}5$

Таблица 2. Сравнение архитектур при $\alpha = 0{,}005$

α	Train loss	Test loss	Test acc., %
0,0005	$0{,}328 \pm 0{,}033$	$0{,}324 \pm 0{,}052$	$89{,}3 \pm 4{,}4$
0,001	$0{,}238 \pm 0{,}026$	$0{,}228 \pm 0{,}030$	$93{,}6 \pm 1{,}8$
0,005	$0{,}113 \pm 0{,}014$	$0{,}105 \pm 0{,}010$	$96{,}8 \pm 0{,}9$
0,01	$0{,}080 \pm 0{,}009$	$0{,}083 \pm 0{,}009$	$96{,}8 \pm 1{,}0$
0,05	$0{,}027 \pm 0{,}003$	$0{,}068 \pm 0{,}011$	$96{,}7 \pm 1{,}1$
0,1	$0{,}011 \pm 0{,}001$	$0{,}073 \pm 0{,}013$	$96{,}8 \pm 1{,}0$

Таблица 3. Влияние скорости обучения на архитектуру `[30, 64, 32, 2]`

Выводы по экспериментам. Во-первых, на этих данных классы почти линейно разделимы: даже сеть без скрытых слоёв (по сути, логистическая регрессия) даёт около 97 % и в пределах разброса не уступает глубоким сетям; скрытые слои нужны скорее по условию задания, чем по необходимости. Во-вторых, при больших сетях потери на обучении падают быстрее, чем на проверке. В-третьих, малая скорость обучения ($\alpha \leq 0{,}001$) за 500 эпох не успевает сойтись, а при $\alpha \geq 0{,}05$ потери на проверке перестают улучшаться при почти нулевых потерях на обучении, что говорит о начале переобучения. Значение $\alpha = 0{,}005$— консервативный выбор, при котором обучение устойчиво, а $\alpha = 0{,}01$–$0{,}05$ дали бы ту же точность быстрее.

Важно, что средняя точность по 10 запускам (около 96,8 %) ниже 98,25 % из таблицы 1: приведённый запуск оказался удачным по начальной инициализации. Разброс ± 1 п. п. сопоставим с ценой одной ошибки на 114 объектах, поэтому различия в десятые доли процента между конфигурациями статистически незначимы.

ВЫВОД

В ходе проекта реализован с нуля на `numpy` многослойный персептрон с настраиваемой архитектурой, ReLU в скрытых слоях, softmax на выходе, инициализацией Хе, функцией потерь— кросс-энтропией и обучением методом обратного распространения ошибки с полнопакетным градиентным спуском. Корректность градиентов подтверждена численной проверкой (относительная ошибка порядка 10^{-9}).

Модель `[30, 64, 32, 2]` достигла 98,25 % точности и бинарной кросс-энтропии 0,0845 на отложенной выборке, ошибившись на 2 объектах из 114. Эксперименты с гиперпараметрами показали, что в среднем качество составляет около 97 % и задача решается уже линейной моделью, а выбор скорости обучения влияет на сходимость заметно сильнее, чем глубина сети.